



Alma Mater Studiorum-Università di Bologna Scuola di Ingegneria

Data classes & Record

Da switch a match/when & pattern matching

Corso di Laurea in Ingegneria Informatica
Anno accademico 2023/2024

Prof. ENRICO DENTI

Dipartimento di Informatica – Scienza e Ingegneria (DISI)



LE CLASSI «DATI»

- Nel modello di un sistema vi sono spesso classi che costituiscono **semplici agglomerati di dati immutabili**
 - campi dati + costruttore primario + accessor
 - **equals**, **hashCode** e **toString** «standard»
- La loro struttura è «meccanica» e standard, al punto che si fanno generare automaticamente allo strumento
 - vedi generazione automatica in Eclipse
- Ma... se sono così standard, ha senso doversi appoggiare allo strumento per fargli generare l'ovvio?
 - *..non potrebbe pensarci il compilatore, magari a partire da una dichiarazione molto semplificata..?*



LE CLASSI «DATI»

- In **Scala** (prima), **Kotlin** (poi) e infine anche in **Java** (15+) è stato introdotto a tale scopo un **costrutto semplificato**
 - **IDEA**: definire i campi dati e lasciare al compilatore «tutto il resto»
 - Scala: case classes
 - Kotlin: data classes
 - Java: record
- L'espressività varia da un linguaggio all'altro
 - in **Scala** le case classes lavorano in tandem col costrutto **match** (evoluzione dello **switch**) che supporta *pattern matching*
 - in **Kotlin** le data classes lavorano in tandem col costrutto **when** (evoluzione dello **switch**) ma fino a un certo punto...
 - in **Java** i record beneficiano del *pattern matching* nel costrutto **switch** (da Java 17, per ora ancora con qualche limitazione)



CASE CLASSES IN SCALA

- In **Scala**, una *case class* – introdotta dalla keyword **case** davanti a **class** – implica le seguenti specificità:
 - la classe contiene dati immutabili
 - le istanze vengono create *senza usare l'operatore new* (è definito un factory method implicito)
 - la classe include definizioni implicite e «standard» per **toString**, **equals**, **hashCode** (in particolare, **equals** confronta i campi)
 - è fornito un metodo **copy** per duplicare un'istanza
 - *la classe supporta l'ereditarietà, ma non su altre case classes* (case-to-case classes inheritance is prohibited)
 - la limitazione si bypassa studiando una diversa architettura software
 - *è supportata la de-strutturazione in forma di pattern matching*
 - si basa sull'uso «behind the scenes» dei metodi accessor



CASE CLASSES: ESEMPIO

```
case class Persona(val nome:String, val anni:Int){}
```

Scala

```
def main(args: Array[String]) : Unit = {
```

Scala

```
    val p1 = Persona("John", 25); println(p1);
```

toString generata automaticamente

```
    val p2 = Persona("Mary", 22); println(p2);
```

```
    val p3 = Persona("John", 25); println(p3);
```

equals generata automaticamente

```
    print(p1.equals(p3)); print(", "); println(p1 == p3);
```

```
    print(p1.hashCode().equals(p3.hashCode()));
```

```
        print(", "); println(p1.hashCode() == p3.hashCode());
```

hashCode generata automaticamente

```
}
```

```
Persona(John,25)
```

```
Persona(Mary,22)
```

```
Persona(John,25)
```

```
true, true
```

```
true, true
```



CASE CLASSES: ESEMPIO

```
case class Persona(val nome:String, val anni:Int){}
```

Scala

```
def main(args: Array[String]) : Unit = {
```

Scala

```
    val p1 = Persona("John", 25); println(p1);
```

```
    val p2 = Persona("Mary", 22); println(p2);
```

```
    val p3 = Persona("John", 25); println(p3);
```

```
    print(p1.equals(p3)); print(", "); println(p1 == p3);
```

```
    print(p1.hashCode().equals(p3.hashCode()));
```

```
        print(", "); println(p1.hashCode() == p3.hashCode());
```

```
    val Persona(nome, età) = p1
    println(nome); println(età);
```

WOW! Pattern matching con
de-strutturazione automatica
dell'oggetto nei componenti! 😊

```
}
```

John

25

CASE CLASSES: ESEMPIO

```
case class Persona(val nome:String, var anni:Int=0) {
```

Scala

```
def show(): String = {
```

```
  this match {
```

```
    case Persona(nome, età) if età<18 => s"$nome è minorenne"
```

```
    case Persona(nome, età) if età<70 => s"$nome è un lavoratore"
```

```
    case other => s"$nome è pensionato"
```

```
  }
```

```
}
```

```
}
```

In Scala, il **costrutto match** (sostituisce lo switch) è un'espressione, non un'istruzione!

Quindi, può restituire un risultato!

Le **guardie semantiche** consentono di discriminare anche semanticamente (non solo sintatticamente) i diversi casi

- Il costrutto **match** di Scala è *molto più* di uno **switch**
 - è un'espressione, non un'istruzione
 - i casi sono mutuamente esclusivi (niente più *fall through*!)
 - supporta pattern matching sintattico e guardie semantiche



CASE CLASSES: ESEMPIO

```
def main(args: Array[String]) : Unit = {  
  
    val p1 = Persona("John", 25); println(p1);  
    val p2 = Persona("Mary", 22); println(p2);  
    println(p1.show());  
    println(p2.show());  
  
    println(Persona("Henry", 55).show());  
    println(Persona("Jane", 17).show());  
    println(Persona("Sergio", 76).show());  
  
}
```

Scala

John è un lavoratore
Mary è un lavoratore
Henry è un lavoratore
Jane è minorenne
Sergio è pensionato



DATA CLASSES IN KOTLIN

- In **Kotlin**, una *data class* – introdotta dalla keyword **data** davanti a **class** – implica le seguenti specificità:
 - la classe contiene dati immutabili
 - la classe include definizioni implicite e «standard» per **toString**, **equals**, **hashCode** (in particolare, **equals** confronta i campi)
 - la classe include *N* metodi accessor **componentN**, uno per campo
 - è fornito un metodo **copy** per duplicare un'istanza
 - una *data class* è **final** e quindi non può neppure essere astratta
 - si può però derivare una data class da una classe standard
 - è supportata la de-strutturazione in forma di *pattern matching*
 - si basa sull'uso «behind the scenes» dei metodi **componentN**



DATA CLASSES: ESEMPIO

```
data class Persona(val nome:String, var anni:Int=0) {}
```

Kotlin

```
fun main(args: Array<String>) : Unit {
```

Kotlin

```
    val p1 = Persona("John", 25); println(p1);
```

toString generata automaticamente

```
    val p2 = Persona("Mary", 22); println(p2);
```

```
    val p3 = Persona("John", 25); println(p3);
```

equals generata automaticamente

```
    print(p1.equals(p3)); print(", "); println(p1 == p3);
```

```
    print(p1.hashCode().equals(p3.hashCode()));
```

```
        print(", "); println(p1.hashCode() == p3.hashCode());
```

hashCode generata automaticamente

```
}
```

```
Persona(nome=John, anni=25)
```

```
Persona(nome=Mary, anni=22)
```

```
Persona(nome=John, anni=25)
```

```
true, true
```

```
true, true
```



DATA CLASSES: ESEMPIO

```
data class Persona(val nome:String, var anni:Int=0) {}
```

Kotlin

```
fun main(args: Array<String>) : Unit {
```

Kotlin

```
    val p1 = Persona("John", 25); println(p1);
```

```
    val p2 = Persona("Mary", 22); println(p2);
```

```
    val p3 = Persona("John", 25); println(p3);
```

```
    print(p1.equals(p3)); print(", "); println(p1 == p3);
```

```
    print(p1.hashCode().equals(p3.hashCode()));
```

```
        print(", "); println(p1.hashCode() == p3.hashCode());
```

```
    val (nome, età) = p1
```

```
    println(nome); println(età);
```

```
}
```

John

25

Pattern matching con
de-strutturazione automatica
dell'oggetto nei suoi componenti! 😊



DATA CLASSES: ESEMPIO

```
data class Persona(val nome:String, var anni:Int=0) {
```

Kotlin

```
fun show(): String {
```

```
    return when(this.anni) {
```

```
        in 0..17 -> "$nome è minorenne"
```

```
        in 18..70 -> "$nome è un lavoratore"
```

```
        else -> "$nome è pensionato"
```

```
    }
```

```
}
```

```
}
```

In Kotlin, il **costrutto** *when* (sostituisce lo switch) è un'espressione, non un'istruzione!
Quindi, può restituire un risultato!

Kotlin supporta solo alcuni *tipi prestabili* di guardie semantiche: *in range* è una

- Il costrutto **when** di Kotlin è *più* di uno **switch** (ma meno del **match**)
 - è un'espressione, non un'istruzione
 - i casi sono mutuamente esclusivi (niente più *fall through*!)
 - supporta pattern matching sintattico, ma non molte guardie semantiche

DATA CLASSES: ESEMPIO

```
data class Persona(val nome:String, var anni:Int=0) {
```

Kotlin

```
    fun show(): String =  
        when(this.anni) {  
            in 0..17 -> "$nome è minorenne"  
            in 18..70 -> "$nome è un lavoratore"  
            else -> "$nome è pensionato"  
        }  
}
```

In generale, in Kotlin se la funzione è costituita da una *singola espressione* si possono evitare sia il blocco `{ }` sia la keyword `return`: basta un `=`

Ulteriori dettagli sul **costrutto `when`**

- un'altra guardia possibile è **`is tipo`**, che verifica che l'oggetto sia del tipo specificato
- usato senza argomenti (**`when{ .. }`**), equivale a una catena di **`if`**

- Il costrutto **`when`** di Kotlin è *più* di uno **`switch`** (ma meno del **`match`**)
 - è un'espressione, non un'istruzione
 - i casi sono mutuamente esclusivi (niente più *fall through*!)
 - supporta pattern matching sintattico, ma con poche guardie semantiche



DATA CLASSES: ESEMPIO

```
fun main(args: Array<String>) : Unit {  
  
    val p1 = Persona("John", 25); println(p1);  
    val p2 = Persona("Mary", 22); println(p2);  
    println(p1.show());  
    println(p2.show());  
  
    println(Persona("Henry", 55).show());  
    println(Persona("Jane", 17).show());  
    println(Persona("Sergio", 76).show());  
  
}
```

Kotlin

John è un lavoratore
Mary è un lavoratore
Henry è un lavoratore
Jane è minorenne
Sergio è pensionato



RECORD IN JAVA 15+

- In **Java** un *record* – introdotto dalla keyword **record** in sostituzione di *class* – implica le seguenti specificità:
 - la classe (record) contiene dati immutabili:
«Records are intended to be simple data carriers»
 - le istanze vengono create normalmente con l'operatore **new**
 - la classe include definizioni implicite e «standard» per **toString**, **equals**, **hashCode** (in particolare, **equals** confronta i campi)
 - la classe include *N* metodi *accessor di nome uguale al campo-dati*
 - è fornito un metodo **copy** per duplicare un'istanza
 - **un record è final e non può essere astratto, né esteso per ereditarietà** (sottobanco infatti estende già la classe **Record..**)
 - da Java 19 è *parzialmente supportata* la de-strutturazione in forma di pattern matching → JEP 427: <https://openjdk.java.net/jeps/427>

RECORD: ESEMPIO 1

```
public record Persona(String nome, int anni) {}
```

Java

```
public static void main(String[] args) {
    var p1 = new Persona("John", 25); System.out.println(p1);
    var p2 = new Persona("Mary", 22); System.out.println(p2);
    var p3 = new Persona("John", 25); System.out.println(p3);
    System.out.print(p1.equals(p3));
    System.out.print(", "); System.out.println(p1 == p3);
    System.out.println(p1.hashCode() == p3.hashCode());
    System.out.println(p1.nome());
    System.out.println(p1.anni());
}
```

toString generata automaticamente

Java

equals generata automaticamente

Accessor generati automaticamente

NB: equals non è ==

```
Persona[nome=John, anni=25]
Persona[nome=Mary, anni=22]
Persona[nome=John, anni=25]
true, false
John
25
```

NB: in Java, l'operatore == non è sinonimo di equals !

Per ora, Java *non ha* pattern di de-strutturazione

hashCode generata automaticamente



RECORD: ESEMPIO 1

```
public record Persona(String nome, int anni) {
```

Java

```
    public String show() {
```

```
        return
```

```
            switch (this.anni) {
```

```
                case 0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16,17
```

```
                    -> this.nome + " è minorenne";
```

```
                case 71,72,73,74,75,76,77,78,79,80,81,82,83,84,85,
```

```
                    86,87,88,89,90,91,92,93,94,95,96,97,98,99,100
```

```
                    -> this.nome + " è un pensionato";
```

```
                default -> this.nome + "è un lavoratore";
```

```
            };
```

```
        }
```

```
    }
```

Da Java 13, la *switch expression* – che può restituire un risultato – si affianca al tradizionale switch statement (che resta).

Nella *switch expression* l'operatore *->* sostituisce il tradizionale :
Grazie al suo *break implicito*, evita il fall-through fra i casi.
Inoltre, tutti i casi devono essere coperti (niente buchi!)

Fino a Java 16, lo *switch* di Java **non** supportava pattern sintattici (es. range di valori) né, a maggior ragione, guardie semantiche



RECORD & PATTERN MATCHING

- Il pattern matching (solo con i record) è stato introdotto come *preview feature* inizialmente in Java 17, con una *sintassi preliminare* basata sull'uso dell'operatore `&&`
- In Java 19, è stato affinato con *modifica sostanziale della sintassi* e introduzione della nuova keyword **when**
 - all'epoca era ancora una *preview feature*
 - raffinata poi in Java 20 (sempre come *preview feature*)
- Da Java 21 è una **feature stabile del linguaggio**
 - la versione finale della sintassi permette anche l'uso di *null* come tipo di guardia semantica, che prima non era supportato



PATTERN MATCHING EVOLUTO

ESEMPIO 2

```
public record Persona(String nome, int anni) {
```

Java

```
    public String show() {
```

```
        return
```

```
        switch(Integer.valueOf(this.anni)) {
```

```
            case Integer i when i<18 ->
```

```
                this.nome + " è minorenne";
```

```
            case Integer i when i>70 ->
```

```
                this.nome + " è un pensionato";
```

```
            default -> this.nome + " è un lavoratore";
```

```
        };
```

```
    }
```

```
}
```

Da Java 19 (preview feature),
le *guardie semantiche* sono
invece introdotte da **when**

In Java 21 sono disponibili anche i
pattern di destrutturazione → next slide



RECORD: ESEMPIO 2

```
public static void main(String[] args) {  
    var p1 = new Persona("John", 25); System.out. println(p1);  
    var p2 = new Persona("Mary", 22); System.out. println(p2);  
  
    System.out.println(p1.show());  
    System.out.println(p2.show());  
  
    System.out.println(new Persona("Henry", 55).show());  
    System.out.println(new Persona("Jane", 17).show());  
    System.out.println(new Persona("Sergio", 76).show());  
}
```

Java

John è un lavoratore
Mary è un lavoratore
Henry è un lavoratore
Jane è minorenne
Sergio è un pensionato



DESTRUCTURING PATTERN negli switch

- Il **destructuring pattern** è offerto in forma stabile da Java 21
 - per ora, ammessa solo entro `instanceof` e `switch`
 - in futuro probabilmente anche nel `for` e in altri costrutti

```
static void print(Object obj) {  
    System.out.println(switch(obj) {  
        case Persona(String n, int a) -> "(" + n + ", " + a + ")";  
        default -> obj.toString();  
    });  
}  
  
public static void main(String[] args) {  
    print(new Persona("John", 25));  
    print(33);  
}
```

Java

Anche nella forma con *type inference*:
Persona(var n, var a)

(John, 25)
33



DESTRUCTURING PATTERN in instanceof

- Il destructuring pattern è offerto in forma stabile da Java 21
 - per ora, ammessa solo entro `instanceof` e `switch`
 - in futuro probabilmente anche nel `for`, e in altri costrutti
- <https://www.youtube.com/watch?v=L6BS2F0dWXQ&t=4486s>

```
static void print2(Object obj) {  
    if (obj instanceof Persona(String n, int a)) {  
        System.out.println("(" + n + ", " + a + ")");  
    }  
    else System.out.println(obj);  
}  
  
public static void main(String[] args) {  
    print2(new Persona("John", 25));  
    print2(33);  
}
```

Java

Anche nella forma con *type inference*:
`Persona(var n, var a)`

(John, 25)
33



JAVA 21: UNNAMED PATTERNS

- Java 21 offre come *preview feature* gli *unnamed patterns*
 - per ora, sempre solo entro `instanceof` e `switch`
 - si può non mettere il nome di una variabile che non si usa
 - Eclipse 2023-12 non li supporta: usare JDK da riga di comando!

```
static void printAge(Object obj) {  
    if (obj instanceof Persona(_, int a)) {  
        System.out.println("età: " + a);  
    }  
    else System.out.println(obj);  
}
```

Java

Anche nella forma con *type inference*:
`Persona(_, var a)`

```
public static void main(String[] args) {  
    printAge(new Persona("John", 25));  
    printAge(33);  
}
```

età: 25
33



DESTRUCTURING PATTERN INNESTATI

- Il destructuring pattern opera anche con **oggetti innestati**:

```
record Rectangle(Point a, Point b) {}  
  
static void print(Rectangle r) {  
    if (r instanceof  
        Rectangle(Point(var x1, var y1), Point(var x2, var y2))) {  
        System.out.println("vertici: (" + x1 + ", " + y1 + "),  
                            (" + x2 + ", " + y2 + ")");  
    }  
    else System.out.println(r);  
}  
  
public static void main(String[] args) {  
    print(new Rectangle(new Point(1,1), new Point(4,4)));  
    print(new Rectangle(new Point(1,1), new Point(-4,4)));  
}
```

Java

Da Java 19, anche con
oggetti innestati

```
vertici: (1,1), (4,4)  
vertici: (1,1), (-4,4)
```



RECORD: DETTAGLI & EREDITARIETÀ

Java

- Un **record** può:
 - introdurre nuovi metodi (dopo tutto, è una normale classe)
 - prevedere campi statici e metodi statici (dopo tutto è una classe)
 - implementare interfacce (capiremo a suo tempo..)
 - definire costruttori ausiliari, ma solo col corpo della forma **this (...)**
 - **ridefinire il costruttore primario, anche in forma shortcut senza argomenti:**
questo schema si usa spesso per introdurre una sezione iniziale di verifica delle precondizioni
- Un **record** non può:
 - **essere esteso in ereditarietà (è una classe **final**), né derivare da altre classi (deriva già da **Record**)**
 - aggiungere ulteriori campi dati di istanza (avrebbero dovuto essere dichiarati nell'intestazione)
 - modificare i campi dati di istanza (sono **final**)

Diverso da Scala (che ammette ereditarietà, anche se le best practice la sconsigliano) e da Kotlin (che la ammette la derivazione da altre classi)



RECORD & COSTRUTTORI ESEMPIO

Java

```
public record Persona(String nome, int anni) {
```

```
    public Persona {
```

```
        // NB: non è un default constructor, non ha lista di argomenti!  
        // E' il preludio del costruttore/2 generato automaticamente
```

```
        java.util.Objects.requireNonNull(nome, "deve esserci un nome");  
        java.util.Objects.requireNonNull(anni, "deve esserci una età");
```

```
    }
```

Questo codice *si aggiunge* a quello del costruttore canonico generato automaticamente

```
    public Persona() { // costruttore ausiliario classico  
        this("Pippo", 333); // no assegnamenti diretti this.xx=xx  
    }
```

```
    ...
```

```
}
```

NB: la classe di libreria **Objects** offre molti metodi di verifica di precondizioni, che consentono anche di specificare azioni alternative in caso esse non siano verificate.

ESPLORARE!



IL CLASSICO CASO

Persone e Studenti

- Viste le limitazioni nell'uso dell'ereditarietà, le classi dati non possono gestire il classico caso Persone/Studenti
 - **Scala** supporta ereditarietà con case classes MA solo fra una classe normale e una case class, NON fra due case classes
 - inoltre, occorre comunque attenzione perché equals ed hashCode non possono sapere se la sottoclasse aggiunge altri dati..
 - **Kotlin** consentirebbe di avere come data class la classe derivata (Studente) ma non la classe base (Persona)
 - **Java** non consente ai record né di estendere, né di essere estesi da altre classi o altri record
 - è però possibile implementare interfacce, anche se ciò comporta riscrivere il codice che non può essere ereditato



IL CLASSICO CASO

Persone e Studenti

```
object MyMain {  
  def main(args: Array[String]) : Unit = {  
    val p1 = Persona("John", 25); println(p1);  
    val s1 = Studente("Henry", 25, 987654); println(s1);  
    println();  
    for (p ← Array(p1,s1,p1,s1)) println(p);  
  }  
}
```

Scala

```
case class Persona(val nome:String, val anni:Int){}
```

Persona è una case class, MA
Studente è una classe normale
→ deve definirsi toString manualmente

```
class Studente(nome:String, anni:Int, val matr:Long) extends Persona(nome, anni){  
  override def toString = "Studente(" + nome + "," + anni + "," + matr + ")";  
}
```

```
class Persona(val nome:String, val anni:Int){  
  override def toString = "Persona(" + nome + "," + anni + ")";  
}
```

Il contrario è meno
conveniente (occorre
definirsi due toString)

```
case class Studente(override val nome:String, override val anni:Int, val matr:Long)  
  extends Persona(nome, anni){  
  override def toString = "Studente(" + nome + "," + anni + "," + matr + ")";  
}
```

IL CLASSICO CASO

Persone e Studenti

```
fun main(args: Array<String>) : Unit {  
    val p1 = Persona("John", 25); println(p1);  
    val s1 = Studente("Henry", 25, 987654); println(s1);  
    println();  
    for (p in arrayOf(p1,s1,p1,s1)) println(p);  
}  
  
open class Persona(open val nome:String, open val anni:Int){  
    override fun toString() = "Persona(" + nome + "," + anni + ")";  
}  
  
data class Studente(override val nome:String, override val anni:Int, val matr:Long)  
    : Persona(nome, anni) {  
    override fun toString() = "Studente(" + nome + "," + anni + "," + matr + ")";  
}
```

Kotlin

Java non consente neppure questo scenario, perché i record non possono né ereditare, né essere ereditati

Persona non può essere una case class, perché Kotlin non ne consentirebbe l'estensione.

Quindi è possibile solo il secondo scenario, in cui Persona è una classe normale e Studente è una data class derivata.

Tuttavia ciò non induce alcun risparmio di codice, perché toString va comunque ridefinita (altrimenti si eredita quella di Persona) → non c'è vero vantaggio